From Findings to Verifiable Remediation Artifacts: A Smart Contract Security Pipeline

Fernando Boiero, Sebastián Norberto Mussetta, and Norberto Gaspar Cena
Universidad Tecnológica Nacional (UTN)
Facultad Regional Villa María
Villa María, Córdoba, Argentina
{fboiero, smussetta, ngcena}@frvm.utn.edu.ar

***Abstract*—Detection tools usually stop at the finding: they identify a vulnerable function, but the patch, regression test, and audit evidence remain manual work. This paper evaluates how far an automated pipeline can move from a finding to inspectable remediation artifacts without hiding residual failures. MIESC takes normalized Solidity findings, emits nearly self-contained patch candidates, generates Foundry exploit tests, creates formal-specification stubs, and maps findings to compliance controls. The corpus-scale evaluation targets patch generation; tests, specifications, and compliance mappings are demonstrated on a single illustrative contract. On SmartBugs-curated (143 contracts), the v2 remediation baseline patches 123 contracts, and all 123 patches compile standalone. The MIESC re-scan reports fewer HIGH/CRITICAL findings for 88 of them, and 121 satisfy a bounded no-regression criterion. Under independent Slither validation, 70 patched contracts are free of HIGH findings (57%; 95% Wilson CI [48%, 65%]). A pre-patch Slither baseline adds independent evidence. Slither's reentrancy detector fires on 28 of 29 analyzable reentrancy contracts before patching and on none after. Among the 121 analyzable patched contracts, those free of HIGH findings rise from 35 to 69 (exact McNemar $p < 10^{-9}$), and nearly all remaining HIGH alerts predate the patch. The artifacts are verifiable in that each can be re-checked independently; the repairs are not proven correct. The pipeline runs locally without API calls and automates the generation of repair candidates for human review, not correct automatic repair. Scripts and per-claim artifacts are released; MIESC is available under AGPL-3.0.**

***Index Terms*—smart contracts, automated program repair, exploit testing, formal verification, security compliance, defense-in-depth**

I. Introduction

The smart contract security ecosystem has invested heavily in *detection*—static analyzers [1], fuzzers [2], symbolic executors [3], and more recently auditors based on large language models (LLMs) [4]—but comparatively little in what happens *after* a vulnerability is found. In practice, the workflow looks like this:

1) A tool reports: "Reentrancy in `withdraw()` (HIGH)."
2) A developer manually writes a patch (add `nonReentrant`).
3) The developer manually writes a test checking the intended fix path.
4) The developer manually maps the finding to compliance requirements.
5) The developer manually writes the audit report.

Steps 2–5 require security expertise that many development teams lack. This paper evaluates the remediation pipeline of MIESC (Multi-layer Intelligent Evaluation for Smart Contracts), which exposes the workflow through composable commands:

```
miesc scan contract.sol -o results.json
miesc fix results.json \
  -c contract.sol -o fixed.sol
miesc test-gen results.json -c contract.sol
miesc compliance results.json --standard mica
miesc report results.json -t premium -f pdf
```

We evaluate patch quality on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories), trace the artifact pipeline on an illustrative vulnerable contract, and compare against five prior automated repair tools. Throughout the paper, *verifiable* refers to the artifacts: each patch, test, specification, and compliance mapping can be re-checked independently. It does not mean that the repairs are proven correct. Likewise, we distinguish the automated *generation of repair candidates*, which is what MIESC provides, from *correct automatic repair*, which our results do not establish. The evidence behind each contribution differs, and we state it explicitly:

1) **Nearly self-contained patch-candidate generation** (`miesc fix`) — *evaluated at corpus scale*. Text-level Solidity patch candidates that avoid adding external dependencies (except a SafeMath directive for pre-0.8 arithmetic). Of the 123 contracts patched under the v2 baseline, 57% (70) are free of HIGH findings under independent Slither validation, the most conservative metric. All 123 patches compile standalone, 72% reduce the HIGH/CRITICAL count on re-scan, and 98% satisfy a bounded no-regression criterion. We foreground the external metric because re-scanning with the detecting pipeline is circular.
2) **Exploit test generation** (`miesc test-gen`) — *demonstrated on one illustrative contract*. Foundry test templates designed to exercise the exploit path on the vulnerable contract and to check that the patched version blocks it; on the illustrative contract, 2 of the 6 tests discriminate between the two versions.
3) **Formal specification generation** (`miesc specs`) — *demonstrated on one illustrative contract*. Certora Verifi-

cation Language (CVL) rules, Scribble annotations, and SMTChecker assertions generated from findings.

4) **Compliance mapping** (`miesc compliance`) — *demonstrated on one illustrative contract*. Rule-based mapping to 12 standard and taxonomy profiles (e.g., ISO 27001, the NIST Cybersecurity Framework (CSF), OWASP, the EU Markets in Crypto-Assets Regulation (MiCA) and Digital Operational Resilience Act (DORA), and the CWE and SWC weakness taxonomies).
5) **Pipeline integration and comparison**: a feature-level comparison against SCRepair, sGuard, Elysium, ACFIX, and SCPatcher, showing which remediation artifacts MIESC emits beyond repair-only baselines.

The contribution is therefore one of pipeline integration rather than a new standalone repair technique. Unlike prior repair tools that stop at the patch, MIESC emits patches, exploit tests, formal-specification stubs, and compliance mappings as inspectable artifacts from a single pipeline. From a technology-transfer standpoint, it targets the practical gap development teams face after detection—turning a finding into a reviewable patch, test, and audit artifact inside existing continuous integration and delivery (CI/CD) workflows.

Our evaluation yields three main findings:

- **Nearly dependency-free patches remove the compilation bottleneck.** Inlining guards instead of importing libraries (only a pre-0.8 SafeMath directive remains), together with a scan-filtered denominator, takes standalone compilation from 51 failures in 141 patches (v1) to none in 123 (v2).
- **Slither's reentrancy detector falls silent on patched reentrancy contracts.** Running Slither on the original and the patched files, its reentrancy detector fires on 28 of 29 analyzable reentrancy contracts before patching and on none after. Over all 121 analyzable patched contracts, those free of HIGH findings rise from 35 to 69 and HIGH contract–detector pairs fall from 141 to 60. Detector silence is evidence, not proof, of a correct repair, and unchecked low-level calls have no Slither HIGH detector to confirm them.
- **What remains is mostly untargeted, pre-existing issues.** In 50 of the 52 analyzable contracts that are not clean after patching, every remaining HIGH detector was already present before patching (chiefly array-length control, weak randomness and state shadowing); 2 patches introduce a new HIGH alert. Extending the patch templates to these classes is the next lever.

II. Related Work

A. Automated Program Repair for Smart Contracts

SCRepair [5] uses genetic programming to mutate Solidity source code until tests pass, but requires pre-existing test suites. sGuard [6] inserts runtime checks (reentrancy locks, integer overflow guards) at near-100% compilation but does not generate tests to verify the fix. Elysium [7] patches at the bytecode level and reports correctly patching at least 30% more contracts than existing tools, but covers 7 vulnerability types. ACFIX [8] uses LLMs guided by mined role-based access control (RBAC) practices and repairs 94.9% of 118 access-control vulnerabilities, but targets a single vulnerability class. SCPatcher [9] applies LLMs with retrieval-augmented generation (RAG) to general-purpose patching, reporting 81.5% fix rate and 91% compilation.

MIESC differs in three ways: (1) patch candidates are *self-contained* in the sense that they avoid new library imports (the pre-0.8 SafeMath directive is the exception), (2) exploit tests are generated alongside patches to check the vulnerable and patched paths, and (3) the pipeline extends beyond patching to formal specifications and compliance mapping. Section IV-G provides a detailed feature comparison.

B. Test Generation for Smart Contracts

Foundry [10] and Echidna [2] provide fuzzing and property-based testing frameworks, but require developers to write test properties manually. Harvey [11] generates high-coverage test inputs but not exploit-specific test cases.

MIESC's `test-gen` command generates vulnerability-specific exploit tests: a reentrancy finding produces a test with an attacker contract that re-enters the victim, while a selfdestruct finding produces an unauthorized-caller test. The generated artifacts serve two roles: exploit-confirmation tests are designed to exercise the vulnerable behavior on the original contract, and fixed-version regression tests check that the same path is blocked after patching.

C. Compliance Automation

In the repair tools surveyed in this paper, compliance mapping is outside the reported scope. Mapping findings to standards such as ISO 27001, MiCA, and NIST CSF is typically performed manually by auditors during report writing. MIESC automates this with a rule-based mapper covering 12 standard and taxonomy profiles.

D. LLM-Assisted Security Analysis

GPTScan [4] combines GPT-4 with program analysis for logic vulnerability detection but produces no patches. EVM-bench [12] evaluates LLM agents on real audit findings in detect, patch and exploit modes; its patch mode scores agent-written fixes against the benchmark's exploit tests, whereas MIESC emits rule-based patch candidates together with their own tests, specifications and compliance mapping. The SmartBugs framework [13] provides execution infrastructure for tool evaluation but includes no patching capabilities. MIESC builds on these detection advances by extending the pipeline beyond detection into actionable remediation artifacts.

III. Pipeline Architecture

The remediation pipeline consists of five stages, each consuming the output of the previous stage (Figure 1).

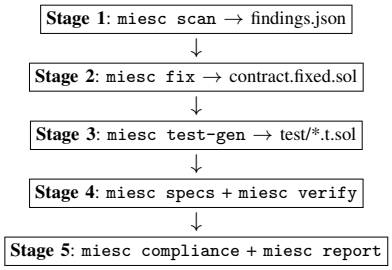


Fig. 1. MIESC remediation pipeline. Each stage consumes the output of the previous stage, producing a progressively more complete security artifact.

A. Stage 1: Detection

Detection is described in our companion paper [14] and is out of scope here; this paper consumes its output. The scanner emits a normalized JSON with findings, severity, location, confidence, canonical category, and remediation fields consumed by the patcher.

B. Stage 2: Patch Generation

`miesc fix` applies text-level patches to the original Solidity source. The patcher operates on the finding's type and function location:

- **Reentrancy**: inserts `nonReentrant` modifier. If OpenZeppelin is not detected, inlines a self-contained `MiescReentrancyGuard` contract (9 lines).
- **Access control / selfdestruct**: inserts `onlyOwner` modifier. If no modifier is defined, inlines a 5-line `onlyOwner` modifier using the existing `owner` state variable.
- **Unchecked calls**: wraps value-bearing low-level calls in `require(success)`.
- **Arithmetic (pre-0.8)**: inserts `using SafeMath for uint256`; unlike the other fixes, this one requires a SafeMath library in the compilation unit, which the 4 patched arithmetic contracts provide.

A deduplication step tracks `(function, fix_category)` pairs to avoid applying the same fix twice when multiple findings target the same function (e.g., `suicidal` and `selfdestruct-identifier` both targeting `destroy()`).

Function name inference: when findings lack an explicit function name (common for LLM-based detections), the patcher infers the nearest function declaration above the reported line number.

1) Patch Example: Reentrancy Guard Injection: Listing 1 shows a vulnerable withdrawal function. The external call on line 3 occurs *before* the state update on line 5, enabling a classic reentrancy attack. Listing 2 shows MIESC's patched version: the inline `MiescReentrancyGuard` (lines 1–9) provides a self-contained mutex that compiles without external dependencies, while the `nonReentrant` modifier on line 14 blocks re-entry. This inline approach avoids requiring OpenZeppelin imports, which is critical for standalone compilation of legacy contracts.

The guard uses only basic state and modifier syntax, so it compiles with every compiler the corpus required (solc 0.4.24–0.5.17); the surrounding example uses 0.8 syntax for readability, and compatibility with 0.6–0.8 code was not evaluated on the corpus. For contracts already inheriting from OpenZeppelin's `ReentrancyGuard`, the patcher detects the existing import and adds only the modifier annotation.

Listing 1. Vulnerable: external call before state update

```
1 function withdraw(uint256 amount) external {
2     require(balances[msg.sender] >= amount);
3     (bool ok,) = msg.sender.call{value: amount}("");
4     require(ok);
5     balances[msg.sender] -= amount;
6 }
```

Listing 2. Patched: inline reentrancy guard injected by MIESC

```
1  contract MiescReentrancyGuard {
2      bool internal _locked;
3      modifier nonReentrant() {
4          require(!_locked, "ReentrancyGuard: reentrant");
5          _locked = true;
6          _;
7          _locked = false;
8      }
9  }
10
11 contract Vault is MiescReentrancyGuard {
12     // ...
13     function withdraw(uint256 amount)
14         external nonReentrant
15     {
16         require(balances[msg.sender] >= amount);
17         (bool ok,) = msg.sender.call{value: amount}("");
18         require(ok);
19         balances[msg.sender] -= amount;
20     }
21 }
```

C. Stage 3: Exploit Test Generation

`miesc test-gen` generates one Foundry test file per finding. Each test is designed with a dual-assertion property:

- **Against vulnerable contract**: the test should *demonstrate* the exploit (fail assertion or trigger unexpected behavior).
- **Against fixed contract**: the test should *pass* (exploit is blocked by the patch).

Templates exist for reentrancy (deploys attacker contract with re-entrant `receive()`), access control and selfdestruct (`vm.prank + vm.expectRevert` for a non-owner caller), unchecked calls (reverting receiver contract), and a generic state-initialization check.

D. Stage 4: Formal Specification

`miesc specs` generates executable specifications from findings:

- **Certora CVL**: a `noReentrancy` rule targeting the vulnerable function.
- **Scribble**: inline annotations (`/// #if_succeeds ...`).
- **SMTChecker**: `assert()` statements encoding the post-patch property (e.g., balance invariants, checked call results).

`miesc verify` executes specifications against available provers (Certora, Halmos, solc SMTChecker).

E. Stage 5: Compliance and Reporting

`miesc compliance` maps each finding to applicable standards using a rule-based mapper with 12 standard and taxonomy profiles. `miesc report` generates audit reports in HTML, PDF, Markdown, or SARIF (Static Analysis Results Interchange Format) with CVSS scores, remediation roadmaps, and executive summaries.

IV. Evaluation

A. Experimental Setup

We evaluate the remediation stage on the SmartBugs-curated corpus [15]: 143 contracts spanning 10 vulnerability categories (reentrancy, access control, unchecked low-level calls, arithmetic, denial of service, front running, short addresses, time manipulation, bad randomness, and other). The remediation metrics and a per-claim trace are generated by the benchmark scripts in the repository. The v2 baseline also runs opt-in Slither validation on patched contracts that compile, recording external HIGH findings separately from the internal MIESC re-scan. Derived artifacts further summarize patch quality by transformation class and standalone compilation failures by category.

Metric definitions. *Fix applied* means the patcher produced a modified source file for at least one fixable HIGH/CRITICAL finding. *Standalone compilation* means the patched file compiles without restoring the original project dependency tree. *Vulnerability eliminated* means that a MIESC re-scan of the patched contract reports fewer HIGH/CRITICAL findings than the scan of the original; it is a count-based criterion, not a check that the specific targeted finding disappeared. *No regression* is a bounded re-scan criterion: the patched contract does not increase the finding count by more than one extra finding (tolerance=1) under the same scanner; it is not a proof of full functional equivalence. Under the strict criterion (tolerance=0, no additional findings permitted) the rate is 115/123 (93%); the reported 98% (121/123) admits the single extra guard-related finding that patch insertion commonly introduces. The full tolerance sensitivity curve (0–10) is released with the artifacts. *External clean-HIGH* means a separate Slither validation run over patched contracts that compile reports no HIGH findings. For the *pre-patch baseline*, we run the same Slither version (0.11.6, as pinned in the repository) with the same compiler per contract on the original files; because one detector proved nondeterministic, each original is analyzed ten times and a detector counts as present if any run reports it. Slither's parser crashes on two original reentrancy contracts (a tuple-assignment construct), although both compile; baseline comparisons therefore cover 121 of the 123 patched contracts. For proportions we report 95% Wilson score intervals, which remain well behaved for the small per-category denominators.

Baselines v1 and v2. Two patcher versions appear in this paper. The v1 run patched 141 contracts, and 51 of those patches failed standalone compilation (42 undefined-symbol and 9 other compiler errors), mostly because guards referenced library symbols that the standalone file did not declare. The v2 patcher makes two changes: (i) it injects *inline* guards and modifiers (e.g., `MiescReentrancyGuard`) instead of relying on imported libraries, and (ii) it computes the denominator from the *current* scan, excluding contracts with no current HIGH/CRITICAL finding (18) or empty scan output (2). All results below refer to v2 unless stated otherwise; v1 figures are given only to explain the change. Table I reconciles the denominators used across sections.

TABLE I
Denominator Reconciliation

n	**Population and where it is used**
143	SmartBugs-curated corpus (all contracts)
142	Original (unpatched) files that compile standalone (validity audit)
141	Contracts patched by the v1 patcher (historical comparison only)
123	v2 patched contracts: $143-18$ no-HIGH -2 empty-scan; denominator of all v2 rates
121	Patched contracts with a valid pre-patch Slither run (Table III)
29	Of these, reentrancy contracts

As an illustrative example, we also trace the full pipeline end-to-end on a single test contract (`AuditTarget.sol`) containing 5 intentional vulnerabilities (reentrancy, missing access control, unprotected selfdestruct, missing zero-address check, and centralized emergency withdrawal). This end-to-end trace is an illustrative walkthrough: the per-stage counts in Table IV and Section IV-F describe this single contract and are not corpus-wide claims.

Environment: macOS ARM64 (Apple Silicon), 48 GB RAM, Foundry v1.3.5. Corpus contracts are compiled with the solc version selected from their pragma (0.4.24–0.5.17); the illustrative contract uses Solidity 0.8.20. The remediation evaluation was re-run after the final detection profile of the companion paper [14]; remediation and detection metrics are kept as separate reproducibility artifacts.

B. Stage 2: Patch Quality

TABLE II
Aggregate Patch Generation Results (v2; 143 contracts, 123 patched)

Metric	**Count**	**Rate**	**95% CI**
Fix applied (current scan)	123/143	86%	[79, 91]
Compilation (standalone)	123/123	100%	[97, 100]
Vuln. eliminated (fewer HIGH)	88/123	72%	[63, 79]
No-regression (tol. = 1)	121/123	98%	[94, 100]
No-regression (tol. = 0)	115/123	93%	[88, 97]
External Slither clean-HIGH	70/123	57%	[48, 65]
Fix failed	0/123	0%	—

Table II summarizes the v2 results. Of the 143 contracts, 123 receive a patch. Every patch compiles standalone. On MIESC re-scan, 88 patched contracts report fewer HIGH/CRITICAL findings than before. Under external Slither validation, 70

patched contracts are clean of HIGH findings; the interval for this conservative metric spans roughly half to two thirds of the patched contracts.

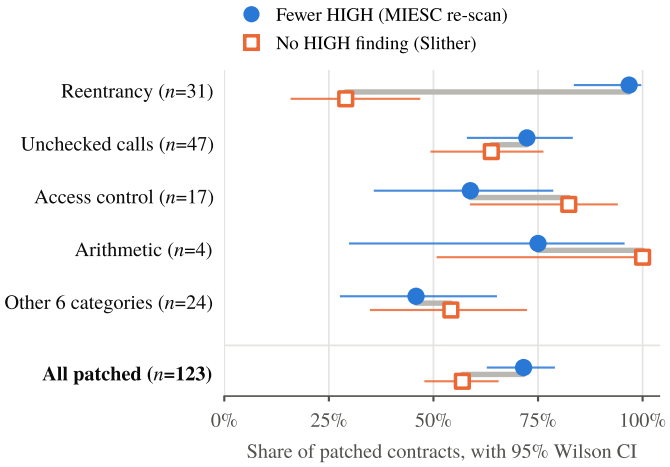


Fig. 2. Self-validation versus independent validation by category. Blue: share of patched contracts for which the MIESC re-scan reports fewer HIGH/CRITICAL findings. Orange: share with no HIGH finding under Slither. All 123 patches compile. The two metrics diverge most for reentrancy (30/31 vs. 9/31) because pre-existing HIGH findings from other detectors remain (Table III). Circles and squares distinguish the series in greyscale.

Fig. 2 breaks the results down by category. The two metrics answer different questions and can diverge sharply. For reentrancy, the MIESC re-scan reports fewer HIGH findings for 30 of 31 patched contracts, yet only 9 are free of every HIGH finding under Slither. The pre-patch baseline (Table III) explains the gap. Slither's own reentrancy detector fires on 28 of 29 analyzable reentrancy contracts before patching and on none afterwards; its 3 remaining reentrancy alerts belong to contracts of other categories. The HIGH alerts left on reentrancy contracts come from other detectors and were already present in the original code. Across all analyzable patched contracts, HIGH contract–detector pairs fall from 141 to 60, and the contracts free of HIGH findings rise from 35 to 69; 34 contracts become clean and none becomes non-clean (exact McNemar $p = 1.2 \times 10^{-10}$). In 50 of the 52 contracts that stay non-clean, every remaining HIGH detector predates the patch. Two patches introduce a new HIGH alert (`arbitrary-send-eth` in one contract, `shadowing-state` in another), which we count as regressions under independent validation. The divergence also runs the other way: for access control, Slither finds more contracts clean (14/17) than the re-scan credits (10/17). Arithmetic has only 4 patched contracts because 11 of the 15 arithmetic contracts produce no current HIGH/CRITICAL finding, and the no-regression criterion fails only for 2 reentrancy contracts.

Experiment validity audit. To separate patch quality from corpus compilability, we additionally compile the original contracts under the same standalone criterion. In the v2 run, 142/143 original files compile standalone. The patcher emits 123 patched contracts; all 123 compile, and all 123 are also checked by external Slither validation without tool errors.

TABLE III
SLITHER HIGH DETECTORS BEFORE AND AFTER PATCHING (NUMBER OF CONTRACTS FLAGGED; 121 ANALYZABLE PATCHED CONTRACTS)

Slither detector	Before	After
`reentrancy-eth`	33	3
`arbitrary-send-eth`	29	2
`msg-value-in-nonpayable`	9	0
`controlled-array-length`	34	28
`weak-prng`	13	13
`shadowing-state`	8	9
Other 6 detectors	15	5
Contract–detector pairs	141	60
Contracts free of HIGH findings	35	69

Before: original contract, Slither 0.11.6; a detector counts if any of ten runs reports it (`encode-packed-collision` fired intermittently on one contract). After: canonical v2 external validation of the patched contract (single run). Script and per-contract results are released.

C. Stage 3: Test Validity

TABLE IV
GENERATED FOUNDRY TESTS (ILLUSTRATIVE CONTRACT, $n = 1$)

Test	Compile	Vuln.	Fixed
Reentrancy (ETH)	✓	Reverts[a]	PASS
Reentrancy (cross-fn)	✓	Reverts[a]	PASS
Selfdestruct (unprotected)	✓	Confirms[b]	PASS
Selfdestruct (identifier)	✓	Confirms[b]	PASS
Unchecked return value	✓	PASS	PASS
Uninitialized state	✓	PASS	PASS
Total	6/6	2/6 discriminate	6/6

Vuln.: exploit-confirmation test on the vulnerable contract. Fixed: fixed-version regression test on the patched contract. [a]The re-entrant call reverts with an arithmetic underflow (Solidity 0.8 checked arithmetic); the patched contract also reverts, at the guard, so these tests do not discriminate. [b]The test expects a revert for a non-owner caller; the call succeeds on the vulnerable contract and reverts on the patched one.

The scan of the illustrative contract yields one test per supported finding, including two findings (unchecked return value, uninitialized state) beyond its five intentional vulnerabilities; the missing-access-control finding is covered by the two selfdestruct tests, which exercise the same unprotected functions. All 6 tests compile with Foundry, and all 6 fixed-version regression tests pass on the patched contract. Only the two selfdestruct tests behave differently on the vulnerable and the patched contract. The two reentrancy tests revert on both versions under Solidity 0.8 (see below), and the other two pass on both, so 4 of the 6 tests do not discriminate between vulnerable and patched code.

Reentrancy exploit test. Listing 3 shows the Foundry test generated by `miesc test-gen` for the reentrancy finding. The test deploys an attacker contract that re-enters `withdraw()` from its `receive()` fallback and asserts that the attacker ends with more than its deposit. Under Solidity 0.8 checked arithmetic, the nested balance decrements underflow and the whole call reverts, so the assertion is never reached; with the inline guard, the first re-entrant call reverts instead. On 0.8 code the test therefore does not observe a drain on either version.

Listing 3. Generated exploit test for reentrancy (simplified)

```
contract Attacker {
    Vault target;
    uint256 count;

    constructor(address _target) {
        target = Vault(_target);
    }

    function attack() external payable {
        target.deposit{value: 1 ether}();
        target.withdraw(1 ether);
    }

    receive() external payable {
        if (count < 5) {
            count++;
            target.withdraw(1 ether);
        }
    }
}

contract ReentrancyTest is Test {
    function test_drain() public {
        Vault vault = new Vault();
        vm.deal(address(vault), 6 ether);
        Attacker atk = new Attacker(
            address(vault));
        vm.deal(address(atk), 1 ether);
        vm.prank(address(atk));
        atk.attack();
        assertGt(address(atk).balance, 1 ether);
    }
}
```

A discriminating reentrancy test for 0.8 code would have to compare outcomes (e.g., expect the drain on the vulnerable contract and a revert on the patched one), or target code whose decrement is unchecked; generating such tests is part of our future work. For each supported category, `test-gen` also emits a fixed-version regression test that expects the exercised path to be blocked after patching.

D. Stage 4: Formal Verification

The formal verification stage applies the Solidity compiler's built-in SMTChecker using the Constrained Horn Clauses (CHC) engine [16]. Unlike fuzzers that generate concrete test inputs, CHC-based verification checks encoded properties over symbolic execution paths via fixed-point computation over the contract's control-flow graph.

Methodology. For each finding, `miesc specs` generates one or more `assert()` statements encoding the property that should hold after patching. The SMTChecker is invoked with `--model-checker-engine chc` and `--model-checker-targets assert,underflow,overflow`. We compare warnings between the original and patched contracts.

Results on illustrative contract. The vulnerable `AuditTarget.sol` produces 2 SMTChecker warnings and 1 compiler warning:

- **Underflow** at line 42 (`balances[msg.sender] -= amount`): reachable when `amount > balances[msg.sender]` after re-entry modifies the balance.
- **Assertion violation** on the generated `assert(address(this).balance >= sum_of_balances)` invariant: violated because re-entry drains more than the sender's legitimate balance.
- **Unchecked low-level call** (compiler warning, not an SMTChecker target): the return value of `.call` is not used.

After patching, the first two warnings disappear entirely (the reentrancy guard makes the re-entry path unreachable), and the third is resolved by the `require(success)` wrapper. On this tractable example, CHC verification provides stronger evidence than the exploit test because it checks the encoded assertion over symbolic executions rather than over one concrete exploit trace.

Why no corpus-wide CHC figure. The CHC engine is driven through the `--model-checker-engine` options of recent compilers (we use 0.8.20); the 0.4.24–0.5.17 compilers that the corpus requires do not offer them, and 142 of the 143 SmartBugs contracts declare a 0.4.x pragma. A corpus-wide CHC measurement would therefore first require porting the contracts to 0.8, which would change the code under test. The evidence for Contribution 3 is limited to the illustrative contract, and a CHC evaluation on a modern 0.8 corpus is future work. This limitation motivates the hybrid approach: exploit tests provide fast executable evidence, while formal verification adds stronger assurance only when the generated specification is tractable.

E. Stage 5: Compliance and Reporting

The compliance mapper assigns each finding to applicable controls using a rule-based mapper with 12 standard and taxonomy profiles. On the illustrative contract, the mapper's rules assign 4 of 11 findings to MiCA, 3 to the NIST CSF 1.1 access-control category PR.AC-4, and 2 to ISO 27001:2022 control A.8.26 (application security requirements). These assignments follow the tool's rule set and are not a legal assessment of regulatory scope. The report generator produces a 3,060-line HTML premium audit report with proof-of-concept (PoC) templates, Common Vulnerability Scoring System (CVSS) v3.1 estimates (base score computation from finding severity and attack vector), and a prioritized remediation roadmap.

F. End-to-End Pipeline (Illustrative)

On the illustrative contract, the scan reports 11 findings (1 critical, 4 high, 4 medium, 2 low). The patcher applies 3 patches (2 further findings are deduplicated onto the same functions), and no HIGH-severity finding remains on re-scan. The 6 generated tests all compile (Table IV details their limits), one SMTChecker specification is generated, 4 findings map to MiCA, and a 3,060-line HTML report is produced. The complete run takes under 10 seconds in local mode with no API cost. These per-stage counts describe one contract and

carry no statistical weight; the corpus-scale evidence is in Table II and Fig. 2.

G. Comparison with Prior Work

TABLE V
FEATURE COMPARISON WITH PRIOR REPAIR TOOLS. T/S/C = GENERATES EXPLOIT TESTS / FORMAL SPECS / COMPLIANCE MAPPING.

Tool	Yr	Vulns	Fix%	Comp.%	T	S/C
SCRepair [5]	'20	GP	—	—	Req[a]	×/×
sGuard [6]	'21	4	—	∼100	×	×/×
Elysium [7]	'22	7 SWC	+30[b]	—	×	×/×
ACFIX [8]	'25	1	94.9	—	×	×/×
SCPatcher [9]	'26	Gen.	81.5	91	×	×/×
MIESC	'26	4 fix cl.[d]	72[c]	100	✓	✓/✓

GP: genetic programming. [a]Requires pre-existing test suite. [b]More contracts correctly patched than existing tools. [c]Each tool's own definition and scope; for MIESC, HIGH/CRITICAL count reduced on re-scan. Not directly comparable; shown for context, not ranking. [d]Four template classes applied across the 10 corpus categories.

Table V compares MIESC against five prior automated repair tools. MIESC reaches 100% standalone compilation for emitted patches (123/123) under a strict no-external-dependency criterion, while SCPatcher reports 91% compilation under different evaluation targets and dependency assumptions, so the numbers are not directly interchangeable; the MIESC "Fix%" column reports the share of patched contracts with fewer HIGH/CRITICAL findings on re-scan (88/123, 72%). ACFIX reports 94.9% fix rate but is limited to a single vulnerability class (RBAC). Of the five compared tools, MIESC is the only one that emits artifacts across the remediation lifecycle (patch generation, exploit testing, formal specifications, and compliance mapping), with the latter three demonstrated on one contract.

H. Failure Analysis

In the v2 baseline, standalone compilation is no longer a failure mode: all 123 emitted patches compile (versus 51/141 failures in v1; see Section IV-A). The remaining v2 gaps are therefore semantic rather than syntactic: 35/123 patched contracts show no reduction in HIGH/CRITICAL findings on MIESC re-scan, and external Slither validation finds only 70/123 patched contracts clean of HIGH (57%), with 53 contracts still reporting HIGH findings.

This reframes the root-cause direction for future work. With standalone compilation solved, the priority shifts from symbol-resolution repair to semantic adequacy: removing the targeted flaw without introducing new HIGH findings under independent validation. The external-validation gap (53 contracts with residual HIGH findings; in 50 of the 52 analyzable ones every remaining detector predates the patch) points to the main lever: extending the templates to `controlled-array-length`, `weak-prng` and `shadowing-state`.

V. DISCUSSION

A. Limitations of Text-Level Patching

MIESC's patcher operates at the text level rather than on the abstract syntax tree (AST). This design choice favors portability (no AST tooling or compiler-specific front end is required) at the cost of precision. Specifically:

1) **No code restructuring**: the patcher cannot reorder statements to implement the Checks-Effects-Interactions (CEI) pattern—the gold standard for reentrancy prevention. Instead, it adds a mutex guard, which is a conservative mitigation but stylistically different from what a human auditor would write.
2) **Regex-based function matching**: unusual formatting (multi-line signatures, Unicode comments, inline assembly, or comments that contain code) can cause mismatched insertions. In the v2 run this risk did not appear as standalone compilation failure for emitted patches, but it remains a design risk for repositories with more complex formatting and dependencies.
3) **Single-contract scope**: cross-contract vulnerabilities (e.g., a proxy's `delegatecall` to a malicious implementation) cannot be patched because the fix requires coordinating changes across multiple files.

These limitations define the tradeoff behind the design. Text-level patching gives broad current-scan coverage across the corpus compilers (0.4.24–0.5.17) and, in the v2 run, 100% standalone compilation for emitted patches. AST-level approaches like sGuard can be more precise, but they usually depend on narrower compiler and AST assumptions.

B. Test Coverage and Correctness

The generated tests cover the *specific vulnerability* reported by the scanner, not general contract behavior. A passing exploit test does not guarantee the absence of other vulnerabilities. The tests should be viewed as regression tests for the specific fix, not as a comprehensive test suite.

Importantly, the exploit tests serve a dual role: (1) when they discriminate, they provide executable evidence for the detection (a test that cannot trigger the vulnerability is a prompt to re-examine the finding, although, as the reentrancy case shows, a real flaw may also be untriggerable under 0.8 checked arithmetic), and (2) they check the patched path exercised by the exploit (if the same exploit still succeeds after patching, the fix is incomplete). None of the repair tools in our comparison reports this kind of bidirectional check.

C. Cost

The evaluated patcher is rule-based and runs locally with no API cost. LLM providers enter only through the detection stage, where local models avoid API costs and frontier providers can be reserved for disputed HIGH/CRITICAL findings. Automated patches are suggestions for review, not changes to merge unreviewed.

D. Towards Continuous Security

The illustrative pipeline's sub-10-second execution time suggests a practical CI/CD integration pattern. A GitHub Action running `miesc scan --ci` on every pull request, combined with `miesc fix --dry-run` to suggest patches

in PR comments, would provide continuous security feedback without blocking development velocity. This integration pattern—detection as a gate, remediation as a suggestion—preserves developer autonomy while making security findings visible earlier in the development process.

E. Threats to Validity

We group seven threats to validity into four categories:

Internal validity. (1) *Scan non-determinism*: the detection scans that feed the patcher come from the companion paper's reproducible profile and are stored as artifacts; 2 scans were retried after empty output, and re-running them may yield slightly different findings. (2) *Re-scan verification*: the fewer-HIGH criterion is measured by re-scanning the patched contract with the same pipeline family that detected the original vulnerability, so a false negative on re-scan would inflate that rate. The v2 baseline mitigates this by adding external Slither validation over every patched contract that compiles and a pre-patch Slither baseline, reporting both separately from the MIESC re-scan rather than merging them into a single success metric. Slither also contributes to MIESC's own static layer, so this validation is external to the re-scan rather than fully independent of the detection pipeline; the before/after comparison is still informative because the patcher never edits code to silence a detector. For reentrancy, Slither's own detector flags 28 of 29 analyzable contracts before patching and none after.

External validity. (3) *Corpus selection bias*: SmartBugs-curated consists mostly of small legacy contracts with known vulnerabilities (142 of 143 declare a 0.4.x compiler pragma). These contracts are small (85 non-blank lines on average, computed over the corpus) and heavily studied, whereas production decentralized-finance (DeFi) protocols are typically larger, multi-file and deeply inherited. Results on production code may differ, particularly for the compilation rate (which depends heavily on dependency availability). (4) *Vulnerability distribution*: the corpus over-represents reentrancy (31 contracts) and unchecked low-level calls (52 contracts) relative to their frequency in production code. Categories such as oracle manipulation, maximal extractable value (MEV) and flash-loan attacks, which account for many recent DeFi incidents [17], are absent from SmartBugs.

Construct validity. (5) *Compilation $\neq$ correctness*: a compiling patch is not necessarily semantically correct. The patch could introduce subtle behavioral changes (e.g., a reentrancy guard that blocks legitimate callback patterns). Our no-regression criterion measures only re-scan finding growth under the same pipeline, and the external Slither clean-HIGH metric measures only residual HIGH findings under one independent analyzer; neither proves preservation of all original functional behavior. (6) *Artifact granularity*: test generation, formal verification, and compliance reporting are evaluated end-to-end on an illustrative contract, while patch quality is measured corpus-wide. We therefore do not claim corpus-wide proof coverage for all generated artifacts.

Development-set overlap. (7) *No held-out set*: the patch templates (reentrancy guard, access-control modifier, checked-call rewrite, SafeMath directive) were developed against SmartBugs-curated itself; the evaluation corpus therefore doubles as the development corpus, with no held-out test split. The reported patch-quality rates should be read as in-distribution results. The transforms are standard, category-level rewrites rather than per-contract fixes—and we explicitly avoid detector-silencing edits—but generalization to unseen contracts is not measured here and remains future work.

Residual threats. Threats (3) and (4) are not mitigated for the remediation stage. Our companion paper [14] evaluates detection on business-logic vulnerabilities from real audits, but remediation on modern production contracts remains future work.

VI. Conclusion and Future Work

This paper reports how far the current MIESC remediation pipeline can move after detection, and where it still falls short. On 143 SmartBugs-curated contracts, the v2 baseline patches 123, and all 123 patches compile standalone. The MIESC re-scan reports fewer HIGH/CRITICAL findings for 88 of them, and 121 satisfy the bounded no-regression criterion. Under independent Slither validation, 70 are free of HIGH findings (57%; 95% CI [48%, 65%]); on the 121 contracts with a pre-patch baseline, that count rises from 35 to 69, and Slither's reentrancy detector, which flags 28 of 29 analyzable reentrancy contracts before patching, flags none afterwards. Reporting the external metric separately shows that patch emission and compilation do not imply semantic closure. Among the five prior repair tools we compare (Table V), MIESC is the only one that reports exploit tests, formal specifications, and compliance documentation alongside patches.

The design choice is to keep each stage inspectable. Detection informs patching, patching informs test generation, test generation checks the exercised exploit path, and formal verification can add stronger evidence when the generated specification is tractable. The current evidence supports this as a workflow that automates the generation of repair candidates for human review, not as a claim of correct automatic program repair.

Limitations. The 100% standalone compilation rate applies to emitted patches after current-scan filtering. Test generation, specification generation, and compliance mapping are demonstrated on a single contract, and the generated reentrancy tests do not discriminate on 0.8 code; Sections V-A and V-E discuss the remaining limitations.

Future work. Three directions are planned: (1) *AST-level patching* via Slither's intermediate representation, enabling CEI reordering, cross-contract fixes and templates for the residual detector classes; (2) *corpus-scale evaluation of tests, specifications and compliance mapping* on a modern 0.8 corpus, with discriminating exploit tests and CHC discharge rates; and (3) *production-scale evaluation* on multi-file DeFi protocols with dependency-aware compilation.

MIESC, the evaluation scripts, and the per-claim artifacts are available under AGPL-3.0 at https://github.com/fboiero/MIESC.

Acknowledgment

This work continues research at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The authors thank the Ethereum security community and the developers of Foundry, Slither, and OpenZeppelin.

References

[1] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

[2] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, "Echidna: Effective, usable, and fast fuzzing for smart contracts," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.

[3] B. Mueller, "Smashing Ethereum smart contracts for fun and real profit," in *HITB Security Conference*, 2018.

[4] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

[5] X. L. Yu, O. Al-Bataineh, D. Lo, and A. Roychoudhury, "Smart contract repair," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–32, 2020.

[6] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "sGuard: Towards fixing vulnerable smart contracts automatically," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021.

[7] C. Ferreira Torres, H. Jonker, and R. State, "Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts," in *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. ACM, 2022.

[8] L. Zhang, K. Li, K. Sun, D. Wu, Y. Liu, H. Tian, and Y. Liu, "ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts," *IEEE Transactions on Software Engineering*, vol. 51, no. 9, pp. 2512–2532, 2025.

[9] X. Li, S. Ye, W. Li, and Z. Li, "SCPatcher: Automated smart contract code repair via retrieval-augmented generation and knowledge graph," *arXiv preprint arXiv:2604.00687*, 2026.

[10] Paradigm, "Foundry: Ethereum development toolkit," https://github.com/foundry-rs/foundry, 2025.

[11] V. Wüstholz and M. Christakis, "Harvey: A greybox fuzzer for smart contracts," in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Industry Track*. ACM, 2020.

[12] J. Wang, A. Bigger, X. Xu, J. W. Lin, A. Applebaum, T. Patwardhan, A. Yukseloglu, and O. Watkins, "EVMbench: Evaluating AI agents on smart contract security," *arXiv preprint arXiv:2603.04915*, 2026.

[13] M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.

[14] F. Boiero, S. N. Mussetta, and N. G. Cena, "MIESC: A multi-layer framework for automated smart contract security evaluation," in *IV Simposio Internacional de Blockchain y Ciberseguridad (RIBCi–CYTED 2026)*, ser. Livros de Atas. Porto, Portugal: Edições CEOS.PP, ISCAP, 2026.

[15] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze Solidity smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.

[16] Solidity Team, "SMTChecker and formal verification," Solidity documentation, https://docs.soliditylang.org/en/latest/smtchecker.html, 2026.

[17] L. Zhou, X. Xiong, J. Ernstberger, S. Chaliasos, Z. Wang, Y. Wang, K. Qin, R. Wattenhofer, D. Song, and A. Gervais, "SoK: Decentralized finance (DeFi) attacks," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 2444–2461.